

PROGRAMMATION EN PYTHON

Licence 2 - Mathématiques, Physique et Chimie

Résumé du cours

1. Types et variables. Python est un langage interprété à typage dynamique. Types de base : int, float, complex, str, bool, list, tuple, dict, set.

2. Structures de contrôle.

```

1 if condition:                # conditionnel
2     ...
3 elif autre:
4     ...
5 else:
6     ...
7
8 for i in range(n):           # boucle for
9     ...
10
11 while condition:             # boucle while
12     ...
    
```

3. Fonctions.

```

1 def nom_fonction(param1, param2, default=0):
2     """Docstring."""
3     return resultat
    
```

4. Bibliothèques scientifiques.

- . numpy : tableaux numériques, algèbre linéaire, fonctions mathématiques.
- . matplotlib : tracé de courbes (plt.plot, plt.show).
- . scipy : intégration, optimisation, statistiques.

5. Listes et compréhensions. [f(x) for x in liste if condition] crée une liste en une ligne. Les tableaux NumPy (np.array) permettent des opérations vectorisées (plus rapides que les boucles).

1. Bases du langage

Exercice 1 - Variables et opérations

★

Exercice

Écrire un programme Python qui :

1. Demande à l'utilisateur deux réels a et b .
2. Affiche leurs somme, différence, produit et rapport.
3. Résout l'équation $ax^2 + bx + c = 0$ (en demandant aussi c) en affichant toutes les racines complexes.

```

1 import cmath # pour les racines complexes
2
3 # Partie 1 : operations de base
4 a = float(input("Entrez a : "))
5 b = float(input("Entrez b : "))
6
7 print(f"Somme      : {a} + {b} = {a + b}")
8 print(f"Difference : {a} - {b} = {a - b}")
9 print(f"Produit    : {a} * {b} = {a * b}")
10 if b != 0:
11     print(f"Rapport      : {a} / {b} = {a / b:.4f}")
12 else:
13     print("Division par zero impossible.")
14
15 # Partie 2 : trinome du second degre
16 print("\n- Resolution du trinome ax^2 + bx + c -")
17 c = float(input("Entrez c : "))
18
19 if a == 0:
20     print("Ce n'est pas un trinome (a = 0).")
21 else:
22     delta = b**2 - 4*a*c
23     x1 = (-b + cmath.sqrt(delta)) / (2*a)
24     x2 = (-b - cmath.sqrt(delta)) / (2*a)
25     if delta > 0:
26         print(f"Deux racines reelles : x1 = {x1.real:.4f}, x2 = {x2.real:.4f}")
27     elif delta == 0:
28         print(f"Racine double : x = {x1.real:.4f}")

```

```

29     else:
30         print(f"Racines complexes : x1 = {x1}, x2 = {x2}")
    
```

2. Calcul scientifique avec NumPy et Matplotlib

Exercice 2 - Tracé de fonctions

★

Exercice

Tracer sur le même graphique les fonctions $f(x) = \sin(x)$, $g(x) = \cos(x)$ et $h(x) = \sin(2x)$ pour $x \in [0, 2\pi]$ avec une légende, un titre et des labels d'axes. Ajouter une grille.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  x = np.linspace(0, 2*np.pi, 500)
5
6  plt.figure(figsize=(9, 5))
7  plt.plot(x, np.sin(x), label=r'$\sin(x)$', linewidth=2)
8  plt.plot(x, np.cos(x), label=r'$\cos(x)$', linewidth=2,
9           linestyle='--')
10 plt.plot(x, np.sin(2*x), label=r'$\sin(2x)$', linewidth=2,
11          linestyle=':')
12
13 plt.axhline(0, color='black', linewidth=0.8)
14 plt.xlabel('$x$ (radians)', fontsize=12)
15 plt.ylabel('Valeur', fontsize=12)
16 plt.title('Fonctions trigonometriques', fontsize=14)
17 plt.legend(fontsize=12)
18 plt.grid(True, alpha=0.4)
19 plt.xticks([0, np.pi/2, np.pi, 3*np.pi/2, 2*np.pi],
20            ['0', r'$\pi/2$', r'$\pi$', r'$3\pi/2$', r'$2\pi$'])
21 plt.tight_layout()
22 plt.show()
    
```

`np.linspace(0, 2*np.pi, 500)` génère 500 points régulièrement espacés entre 0 et 2π . Les opérations NumPy (`np.sin`, `np.cos`) s'appliquent vectoriellement à tous les points simultanément.

Exercice 3 - Méthodes numériques

★★

Exercice

1. Implémenter la méthode de la bisection pour trouver la racine de $f(x) = x^3 - x - 2$ sur $[1, 2]$.
2. Implémenter la méthode d'Euler pour résoudre $y' = y \sin(x)$, $y(0) = 1$ sur $[0, 5]$. Comparer avec la solution exacte $y(x) = e^{1-\cos x}$.

1. Méthode de la bisection :

```

1 def bisection(f, a, b, tol=1e-8, max_iter=100):
2     """Trouve la racine de f sur [a,b] par bisection."""
3     if f(a) * f(b) > 0:
4         raise ValueError("f(a) et f(b) doivent etre de signes opposes.")
5     for _ in range(max_iter):
6         c = (a + b) / 2
7         if abs(f(c)) < tol or (b - a) / 2 < tol:
8             return c
9         if f(a) * f(c) < 0:
10            b = c
11        else:
12            a = c
13    return (a + b) / 2
14
15 f = lambda x: x**3 - x - 2
16 racine = bisection(f, 1, 2)
17 print(f"Racine : x = {racine:.8f}") # x ~= 1.52137971

```

2. Méthode d'Euler :

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Methode d'Euler
5 def euler(f, y0, x0, xf, n):
6     h = (xf - x0) / n
7     x = np.linspace(x0, xf, n+1)
8     y = np.zeros(n+1)
9     y[0] = y0
10    for i in range(n):
11        y[i+1] = y[i] + h * f(x[i], y[i])

```

```

12     return x, y
13
14 f = lambda x, y: y * np.sin(x)
15 x_euler, y_euler = euler(f, 1, 0, 5, 200)
16
17 # Solution exacte
18 x_exact = np.linspace(0, 5, 1000)
19 y_exact = np.exp(1 - np.cos(x_exact))
20
21 plt.plot(x_euler, y_euler, label="Euler (n=200)", linestyle='--',
22          ')
23 plt.plot(x_exact, y_exact, label="Solution exacte")
24 plt.legend(); plt.grid(True); plt.show()

```

Exercice 4 - Algèbre linéaire avec NumPy

★★

Exercice

Résoudre le système $A\vec{x} = \vec{b}$ avec :

$$A = \begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 8 \\ -11 \\ -3 \end{pmatrix}$$

en utilisant `numpy.linalg.solve`. Vérifier la solution, calculer $\det(A)$ et les valeurs propres de A .

```

1 import numpy as np
2
3 A = np.array([[2, 1, -1],
4               [-3, -1, 2],
5               [-2, 1, 2]], dtype=float)
6
7 b = np.array([8, -11, -3], dtype=float)
8
9 # Resolution du systeme
10 x = np.linalg.solve(A, b)
11 print(f"Solution : x = {x}") # x = [2, 3, -1]
12
13 # Verification : A @ x doit etre egale a b
14 residuel = np.linalg.norm(A @ x - b)
15 print(f"Residuel ||Ax - b|| = {residuel:.2e}")

```

```

16
17 # Determinant
18 det = np.linalg.det(A)
19 print(f"det(A) = {det:.4f}")
20
21 # Valeurs propres
22 valeurs_propres, vecteurs_propres = np.linalg.eig(A)
23 print(f"Valeurs propres : {valeurs_propres}")
    
```

La solution est $x_1 = 2$, $x_2 = 3$, $x_3 = -1$. `np.linalg.solve` utilise la décomposition LU, ce qui est bien plus efficace que l'inversion de matrice (`np.linalg.inv`).

Exercice 5 - Traitement de données et statistiques

Exercice

On dispose d'une série de mesures de la résistance d'un conducteur à différentes températures :

T (°C)	20	40	60	80	100	120
R (Ω)	10.2	11.1	12.0	12.8	13.9	14.7

1. Calculer la moyenne, l'écart-type et le coefficient de variation de R .
2. Effectuer une régression linéaire $R = aT + b$ par la méthode des moindres carrés (`numpy.polyfit`).
3. Estimer la résistance à $T = 150^\circ\text{C}$.
4. Tracer les données et la droite de régression.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 T = np.array([20, 40, 60, 80, 100, 120], dtype=float)
5 R = np.array([10.2, 11.1, 12.0, 12.8, 13.9, 14.7])
6
7 # 1. Statistiques descriptives
8 print(f"Moyenne de R : {np.mean(R):.3f} Ohm")
9 print(f"Ecart-type : {np.std(R, ddof=1):.3f} Ohm")
10 print(f"Coeff. variation : {100*np.std(R, ddof=1)/np.mean(R):.1f
    } %")
11
12 # 2. Regression lineaire
13 coeffs = np.polyfit(T, R, 1)
14 a, b = coeffs
    
```

```

15 print(f"\nRegression : R = {a:.4f} * T + {b:.4f}")
16
17 # Coefficient de correlation
18 r = np.corrcoef(T, R)[0, 1]
19 print(f"Coefficient de correlation r = {r:.4f}")
20
21 # 3. Estimation a T = 150 degC
22 T_pred = 150
23 R_pred = a * T_pred + b
24 print(f"\nEstimation a {T_pred} degC : R = {R_pred:.2f} Ohm")
25
26 # 4. Trace
27 T_fit = np.linspace(0, 160, 100)
28 R_fit = a * T_fit + b
29
30 plt.scatter(T, R, color='black', zorder=5, label='Mesures')
31 plt.plot(T_fit, R_fit, label=f'Regression: R={a:.3f}T+{b:.3f}')
32 plt.xlabel('Temperature (degC)'); plt.ylabel('Resistance (Ohm)'
33 )
34 plt.title('Resistance vs Temperature')
35 plt.legend(); plt.grid(True); plt.show()

```

Résultats : $a \approx 0,045 \Omega/^{\circ}\text{C}$, $b \approx 9,28 \Omega$, $r \approx 0,9997$. La linéarité est excellente. À 150°C , $R \approx 16,0 \Omega$.

3. Structures de données et algorithmes

Exercice 6 - Listes, tri et compréhension

★

Exercice

1. Créer une liste des carrés des entiers de 1 à 20 en utilisant une compréhension de liste.
2. Filtrer les éléments pairs de cette liste.
3. Trier une liste de mots par longueur (puis alphabétiquement en cas d'égalité) :
`mots = ["chat", "python", "code", "AI", "Numpy", "ax"]`.
4. Écrire une fonction qui recherche un élément dans une liste triée par l'algorithme de *recherche dichotomique*.

```

1 # 1. Compréhension de liste : carres
2 carres = [n**2 for n in range(1, 21)]

```

```

3 print("Carres:", carres)
4
5 # 2. Filtrage des elements pairs
6 pairs = [x for x in carres if x % 2 == 0]
7 print("Carres pairs:", pairs)
8
9 # 3. Tri par longueur puis alphabetique
10 mots = ["chat", "python", "code", "AI", "Numpy", "ax"]
11 mots_tries = sorted(mots, key=lambda m: (len(m), m.lower()))
12 print("Mots tries:", mots_tries)
13 # Resultat: ['AI', 'ax', 'chat', 'code', 'Numpy', 'python']
14
15 # 4. Recherche dichotomique
16 def recherche_dicho(liste, cible):
17     """Recherche dichotomique dans une liste triee.
18     Retourne l'indice si trouve, -1 sinon."""
19     gauche, droite = 0, len(liste) - 1
20     while gauche <= droite:
21         milieu = (gauche + droite) // 2
22         if liste[milieu] == cible:
23             return milieu
24         elif liste[milieu] < cible:
25             gauche = milieu + 1
26         else:
27             droite = milieu - 1
28     return -1
29
30 nombres = sorted([3, 7, 1, 15, 9, 4, 12]) # [1, 3, 4, 7, 9,
31     12, 15]
32 print(f"Recherche de 9 : indice {recherche_dicho(nombres, 9)}")
33     # 4
34 print(f"Recherche de 6 : indice {recherche_dicho(nombres, 6)}")
35     # -1
    
```

Complexité : La recherche dichotomique a une complexité $O(\log_2 n)$ au lieu de $O(n)$ pour la recherche linéaire. Pour $n = 10^6$ éléments, seulement ~ 20 comparaisons suffisent.

Exercice 7 - Fonctions récursives

★★

Exercice

1. Écrire une fonction récursive calculant $n!$ et une version itérative. Comparer pour $n = 10$.
2. Écrire une fonction récursive pour les nombres de Fibonacci $F_n = F_{n-1} + F_{n-2}$, $F_0 = 0$, $F_1 = 1$.
3. Pourquoi la version naïve de Fibonacci est-elle exponentiellement lente ? Implémenter une version mémorisée avec un dictionnaire.
4. Calculer F_{35} avec les deux versions et mesurer le temps avec `time.time()`.

```

1 import time
2
3 # 1. Factorielle
4 def factorielle_rec(n):
5     """Factorielle recursive."""
6     if n <= 1:
7         return 1
8     return n * factorielle_rec(n - 1)
9
10 def factorielle_iter(n):
11     """Factorielle iterative."""
12     result = 1
13     for i in range(2, n + 1):
14         result *= i
15     return result
16
17 print(f"10! (recursif) = {factorielle_rec(10)}")
18 print(f"10! (iteratif) = {factorielle_iter(10)}")
19
20 # 2. Fibonacci naif (exponentiel)
21 def fib_naif(n):
22     if n <= 1:
23         return n
24     return fib_naif(n-1) + fib_naif(n-2)
25
26 # 3. Fibonacci memoize (lineaire)
27 def fib_memo(n, cache={}):
28     if n in cache:
29         return cache[n]
30     if n <= 1:
31         return n
32     cache[n] = fib_memo(n-1, cache) + fib_memo(n-2, cache)

```

```

33     return cache[n]
34
35 # 4. Comparaison de temps
36 n = 35
37 t0 = time.time()
38 res_naif = fib_naif(n)
39 t1 = time.time()
40 print(f"F({n}) naif      = {res_naif} en {t1-t0:.3f} s")
41
42 t0 = time.time()
43 res_memo = fib_memo(n)
44 t1 = time.time()
45 print(f"F({n}) memoize = {res_memo} en {t1-t0:.6f} s")

```

Analyse : La version naïve calcule $\sim 2^{35}$ appels (exponentiel), tandis que la version mémoisée fait $O(n)$ calculs. Pour $n = 35$, la version naïve prend ~ 5 secondes, la mémoisée ~ 0 ms.

Exercice 8 - Lecture et écriture de fichiers

★

Exercice

On dispose d'un fichier texte `mesures.txt` contenant des lignes de la forme `t R` (temps et résistance).

1. Écrire un programme qui génère ce fichier avec des données simulées ($T = [20, 40, 60, 80, 100]$ °C, $R = 10 + 0.05T + \text{bruit}$).
2. Lire le fichier et extraire les données dans deux listes.
3. Calculer la moyenne et l'écart-type de R à partir des données lues.
4. Réécrire un fichier `resultats.txt` avec les statistiques calculées.

```

1 import numpy as np
2
3 # 1. Generation et ecriture du fichier
4 np.random.seed(42)
5 T_vals = [20, 40, 60, 80, 100]
6 R_vals = [10 + 0.05*T + np.random.normal(0, 0.1) for T in
7           T_vals]
8
9 with open("mesures.txt", "w") as f:
10     f.write("# Temperature(C)  Resistance(Ohm)\n")
11     for T, R in zip(T_vals, R_vals):

```

```

11         f.write(f"{T:.1f} {R:.4f}\n")
12
13 print("Fichier mesures.txt cree.")
14
15 # 2. Lecture du fichier
16 temperatures = []
17 resistances = []
18
19 with open("mesures.txt", "r") as f:
20     for ligne in f:
21         if ligne.startswith("#"): # ignorer commentaires
22             continue
23         parts = ligne.strip().split()
24         if len(parts) == 2:
25             temperatures.append(float(parts[0]))
26             resistances.append(float(parts[1]))
27
28 print("Temperatures lues:", temperatures)
29 print("Resistances lues :", [f"{r:.4f}" for r in resistances])
30
31 # 3. Statistiques
32 R_arr = np.array(resistances)
33 moyenne = np.mean(R_arr)
34 ecart_type = np.std(R_arr, ddof=1)
35 print(f"\nMoyenne R = {moyenne:.4f} Ohm")
36 print(f"Ecart-type = {ecart_type:.4f} Ohm")
37
38 # 4. Ecriture des resultats
39 with open("resultats.txt", "w") as f:
40     f.write("=== Resultats statistiques ===\n")
41     f.write(f"Nombre de mesures : {len(resistances)}\n")
42     f.write(f"Moyenne R : {moyenne:.4f} Ohm\n")
43     f.write(f"Ecart-type : {ecart_type:.4f} Ohm\n")
44     f.write(f"Min / Max : {R_arr.min():.4f} / {R_arr.
45         max():.4f} Ohm\n")
46
47 print("\nFichier resultats.txt ecrit.")

```

Exercice 9 - NumPy : tableaux et algèbre linéaire

★★

Exercice

1. Créer un tableau NumPy A de taille 3×3 représentant la matrice de rotation d'angle $\theta = 45^\circ$.
2. Calculer $\det(A)$, A^{-1} , et les valeurs propres et vecteurs propres de A .
3. Résoudre le système $Ax = b$ avec $b = (1, 0, 1)^T$.
4. Vérifier que $A^{-1}A = I$ et que la solution satisfait bien $Ax = b$.

Matrice de rotation 3D autour de z :

$$A = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

```

1 import numpy as np
2
3 theta = np.pi / 4 # 45 degrees
4
5 # 1. Matrice de rotation
6 A = np.array([
7     [np.cos(theta), -np.sin(theta), 0],
8     [np.sin(theta),  np.cos(theta), 0],
9     [0,              0,             1]
10 ])
11 print("Matrice A :")
12 print(np.round(A, 6))
13
14 # 2. Determinant, inverse, valeurs propres
15 det_A = np.linalg.det(A)
16 print(f"\ndet(A) = {det_A:.6f}") # doit valoir 1
17
18 A_inv = np.linalg.inv(A)
19 print("\nA^(-1) :")
20 print(np.round(A_inv, 6))
21
22 vals_propres, vects_propres = np.linalg.eig(A)
23 print(f"\nValeurs propres : {np.round(vals_propres, 4)}")
24 # Pour une rotation, les VP sont 1 et exp(+i*theta)
25
26 # 3. Resolution du systeme
27 b = np.array([1.0, 0.0, 1.0])
28 x = np.linalg.solve(A, b)

```

```

29 print(f"\nSolution x = {np.round(x, 6)}")
30
31 # 4. Verification
32 identite = np.round(A_inv @ A, 10)
33 print("\nA(-1) @ A =")
34 print(identite)
35
36 residuel = np.linalg.norm(A @ x - b)
37 print(f"\nResidu ||Ax - b|| = {residuel:.2e}")

```

Résultats attendus : $\det(A) = 1$ (rotation préserve les volumes), valeurs propres $\{1, e^{i\pi/4}, e^{-i\pi/4}\}$, résidu $\approx 10^{-15}$.

Exercice 10 - Matplotlib : visualisation et histogramme

★★

Exercice

1. Générer $N = 1000$ réalisations d'une variable aléatoire gaussienne $X \sim \mathcal{N}(\mu = 5, \sigma = 2)$ avec NumPy.
2. Tracer l'histogramme normalisé (densité de probabilité) et superposer la densité théorique $f(x) = \frac{1}{\sigma\sqrt{2\pi}}e^{-(x-\mu)^2/(2\sigma^2)}$.
3. Calculer la moyenne et l'écart-type empiriques et les comparer aux valeurs théoriques.
4. Tracer une figure avec deux sous-graphiques : (a) l'histogramme, (b) la courbe de distribution cumulée empirique (ECDF).

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.stats import norm
4
5 # 1. Generation des donnees
6 np.random.seed(42)
7 mu, sigma = 5, 2
8 N = 1000
9 data = np.random.normal(mu, sigma, N)
10
11 # 3. Statistiques empiriques
12 print(f"Moyenne empirique : {data.mean():.3f} (theorique : {mu})")
13 print(f"Ecart-type empirique : {data.std():.3f} (theorique : {sigma})")

```

```

14
15 # 2. et 4. Graphiques
16 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
17
18 # Sous-graphique 1 : Histogramme
19 ax1.hist(data, bins=40, density=True, alpha=0.6,
20          color='steelblue', label=f'Histogramme (N={N})')
21
22 # Densite theorique
23 x_theo = np.linspace(data.min(), data.max(), 200)
24 ax1.plot(x_theo, norm.pdf(x_theo, mu, sigma),
25          'r-', linewidth=2,
26          label=rf'$\mathcal{{N}}(\mu, \sigma)$')
27 ax1.set_xlabel('x'); ax1.set_ylabel('Densite')
28 ax1.set_title('Histogramme normalise')
29 ax1.legend()
30 ax1.grid(alpha=0.3)
31
32 # Sous-graphique 2 : ECDF
33 data_sorted = np.sort(data)
34 ecdf = np.arange(1, N+1) / N
35
36 ax2.step(data_sorted, ecdf, color='steelblue', label='ECDF
    empirique')
37 ax2.plot(x_theo, norm.cdf(x_theo, mu, sigma),
38          'r-', linewidth=2, label='CDF theorique')
39 ax2.set_xlabel('x'); ax2.set_ylabel('P(X <= x)')
40 ax2.set_title('Fonction de repartition')
41 ax2.legend()
42 ax2.grid(alpha=0.3)
43
44 plt.tight_layout()
45 plt.savefig("statistiques.pdf", bbox_inches='tight')
46 plt.show()
    
```

Interprétation : L'histogramme normalisé (densité) converge vers la densité théorique en $1/\sqrt{N}$. L'ECDF (Empirical Cumulative Distribution Function) permet de visualiser la distribution sans choisir une largeur de bin.

4. Algorithmique et structures de données

Exercice 11 - Parité et année bissextile

★

Exercice

1. Écrire un programme qui demande un entier n à l'utilisateur et affiche s'il est pair ou impair.
2. Écrire un programme qui demande une année et indique si elle est bissextile. Rappel : une année est bissextile si elle est divisible par 4 mais pas par 100, sauf si elle est divisible par 400.

```

1 # 1. Test de parite
2 n = int(input("Entrez un entier n : "))
3 if n % 2 == 0:
4     print(f"{n} est pair.")
5 else:
6     print(f"{n} est impair.")
7
8 # 2. Annee bissextile
9 annee = int(input("Entrez une annee : "))
10 if (annee % 4 == 0 and annee % 100 != 0) or (annee % 400 == 0):
11     print(f"{annee} est bissextile.")
12 else:
13     print(f"{annee} n'est pas bissextile.")
    
```

Tests : 2024 (bissextile, divisible par 4 mais pas 100), 1900 (non bissextile, divisible par 100 mais pas 400), 2000 (bissextile, divisible par 400).

Exercice 12 - PGCD, PPCM et décomposition binaire

★★

Exercice

1. Écrire une fonction `pgcd(a, b)` utilisant l'algorithme d'Euclide, puis une fonction `ppcm(a, b)` qui en déduit le PPCM via la relation $a \times b = \text{pgcd}(a, b) \times \text{ppcm}(a, b)$.
2. Écrire une fonction `binaire(n)` qui retourne la représentation binaire (chaîne de caractères) d'un entier positif sans utiliser `bin()`.
3. Tester avec $a = 84$, $b = 60$ et $n = 77$.

```

1 def pgcd(a, b):
2     """Algorithme d'Euclide iteratif."""
3     while b != 0:
4         a, b = b, a % b
    
```

```

5     return a
6
7 def ppcm(a, b):
8     """PPCM via la relation  $a*b = \text{pgcd}(a,b) * \text{ppcm}(a,b)$ ."""
9     if a == 0 or b == 0:
10        return 0
11    return abs(a * b) // pgcd(a, b)
12
13 def binaire(n):
14     """Decomposition binaire d'un entier positif."""
15     if n == 0:
16        return "0"
17    bits = []
18    while n > 0:
19        bits.append(str(n % 2))
20        n //= 2
21    return "".join(reversed(bits))
22
23 # Tests
24 a, b = 84, 60
25 print(f"pgcd({a}, {b}) = {pgcd(a, b)}")    # 12
26 print(f"ppcm({a}, {b}) = {ppcm(a, b)}")    # 420
27
28 n = 77
29 print(f"binaire({n}) = {binaire(n)}")      # 1001101
30 print(f"verification : bin({n}) = {bin(n)[2:]}")

```

Résultats : $\text{pgcd}(84, 60) = 12$, $\text{ppcm}(84, 60) = 420$, $77 = 64 + 8 + 4 + 1 = (1001101)_2$.

Exercice 13 - Permutation et équation du premier degré

★

Exercice

1. Écrire une fonction `permute(a, b)` qui échange les valeurs de deux variables sans utiliser de variable temporaire (astuce arithmétique ou $a, b = b, a$).
2. Écrire un programme qui résout l'équation $ax + b = 0$ (avec a, b saisis par l'utilisateur) en traitant tous les cas : solution unique, infinité de solutions, aucune solution.

```

1 # 1. Permutation sans variable temporaire
2 def permute_arith(a, b):

```



```

3     """Permutation par somme/difference (entiers/reels)."""
4     a = a + b
5     b = a - b
6     a = a - b
7     return a, b
8
9 def permute_python(a, b):
10     """Permutation pythonique (tuples)."""
11     return b, a
12
13 x, y = 5, 12
14 x, y = permute_python(x, y)
15 print(f"Apres permutation : x = {x}, y = {y}")    # x=12, y=5
16
17 # 2. Equation du premier degre a*x + b = 0
18 a = float(input("Coeff a : "))
19 b = float(input("Coeff b : "))
20
21 if a == 0:
22     if b == 0:
23         print("Infinité de solutions (tout réel x convient).")
24     else:
25         print("Aucune solution (équation impossible).")
26 else:
27     x = -b / a
28     print(f"Solution unique : x = {x:.6f}")

```

Remarque. La permutation arithmétique $a=a+b$; $b=a-b$; $a=a-b$ ne fonctionne qu'avec des nombres et peut provoquer des dépassements de capacité dans certains langages; l'échange par tuples $a, b = b, a$ est la méthode idiomatique en Python.

Exercice 14 - Indice et valeur du maximum dans une liste

★

Exercice

Écrire un programme qui :

1. Demande à l'utilisateur n nombres réels et les stocke dans une liste.
2. Détermine la valeur maximale et son indice (première occurrence).
3. Détermine également la valeur minimale et son indice.
4. Affiche la moyenne, la médiane et l'étendue.

```

1  # Saisie de la liste
2  n = int(input("Combien de valeurs ? "))
3  valeurs = [float(input(f"valeur {i+1} : ")) for i in range(n)]
4
5  # Recherche du maximum et de son indice
6  val_max = valeurs[0]
7  idx_max = 0
8  for i, v in enumerate(valeurs):
9      if v > val_max:
10         val_max = v
11         idx_max = i
12
13  # Recherche du minimum et de son indice
14  val_min = valeurs[0]
15  idx_min = 0
16  for i, v in enumerate(valeurs):
17      if v < val_min:
18         val_min = v
19         idx_min = i
20
21  # Statistiques
22  moyenne = sum(valeurs) / n
23  etendue = val_max - val_min
24  valeurs_triees = sorted(valeurs)
25  if n % 2 == 1:
26      mediane = valeurs_triees[n // 2]
27  else:
28      mediane = (valeurs_triees[n//2 - 1] + valeurs_triees[n//2])
29         / 2
30
31  print(f"Maximum : {val_max} a l'indice {idx_max}")
32  print(f"Minimum : {val_min} a l'indice {idx_min}")
33  print(f"Moyenne : {moyenne:.4f}")
34  print(f"Mediane : {mediane}")
35  print(f"Etendue : {etendue}")

```

Note : En Python, on peut aussi utiliser les fonctions natives `max()`, `min()`, `list.index()` et le module `statistics` (fonction `median()`). L'implémentation manuelle reste utile pour comprendre la logique algorithmique.

Exercice 15 - Somme de série géométrique et intégrale numérique

★★

Exercice

1. Calculer la somme partielle $S_n = \sum_{k=0}^n q^k$ pour $q = 0,5$ et $n = 20$, et la comparer à la limite $S_\infty = \frac{1}{1-q}$. Afficher l'erreur relative.
2. Calculer numériquement l'intégrale $I = \int_0^1 e^{-x^2} dx$ par la méthode des rectangles (à gauche) avec $N = 1000$ intervals. Comparer avec la valeur de référence $I \approx 0,746824$.
3. Refaire le calcul avec la méthode des trapèzes et comparer la précision.

```

1 import math
2
3 # 1. Somme de la serie geometrique
4 q = 0.5
5 n = 20
6 S_n = sum(q**k for k in range(n + 1))
7 S_inf = 1 / (1 - q)
8 erreur_rel = abs(S_n - S_inf) / S_inf * 100
9 print(f"S_{n}          = {S_n:.10f}")
10 print(f"S_inf         = {S_inf:.10f}")
11 print(f"Erreur rel. = {erreur_rel:.3e} %")
12
13 # 2. Methode des rectangles (gauche)
14 def f(x):
15     return math.exp(-x**2)
16
17 N = 1000
18 a, b = 0.0, 1.0
19 h = (b - a) / N
20 I_rect = sum(f(a + i * h) for i in range(N)) * h
21 print(f"\nIntegrale (rectangles) : I = {I_rect:.6f}")
22
23 # 3. Methode des trapezes
24 I_trap = (f(a) + f(b)) / 2
25 for i in range(1, N):
26     I_trap += f(a + i * h)
27 I_trap *= h
28 print(f"Integrale (trapezes)      : I = {I_trap:.6f}")

```

```

29
30 I_ref = 0.746824
31 print(f"Valeur de reference      : I = {I_ref:.6f}")
32 print(f"Erreur rectangles : {abs(I_rect - I_ref):.2e}")
33 print(f"Erreur trapezes      : {abs(I_trap - I_ref):.2e}")

```

Résultats : $S_{20} \approx 0,9999990463$ avec une erreur relative de $\sim 5 \times 10^{-6} \%$. Pour l'intégrale, la méthode des rectangles donne $I \approx 0,746980$ (erreur $\sim 10^{-4}$) et la méthode des trapèzes $I \approx 0,746824$ (erreur $\sim 10^{-7}$, plus précise car la méthode des trapèzes est d'ordre 2).